

Sistema Inteligente de Soporte Hospitalario La Salle Health Center

Alumnos:

Roger Jové Tusell y Teo Aranda Páez

1. Descripción del problema

1.1 Contexto del proyecto

El La Salle Health Center es un hospital simulado que necesita modernizar su gestión clínica mediante la introducción de inteligencia artificial y Big Data. La motivación surge de un problema real en el sector sanitario: los radiólogos procesan cientos de radiografías de tórax al día, con el consiguiente riesgo de fatiga visual y retraso en el diagnóstico de patologías urgentes como COVID-19 o neumonía.

Este proyecto ha sido desarrollado como práctica académica en el marco de la asignatura de Programació d'Intel·ligència Artificial, con el objetivo de construir un sistema completo que integre modelos de IA, infraestructura cloud-native, pipeline de datos y una interfaz web funcional.

1.2 Objetivos planteados

Los objetivos principales del proyecto son:

- 1. Desarrollar un modelo de clasificación de radiografías de tórax capaz de distinguir entre tres categorías: sana, neumonía y COVID-19.**
- 2. Diseñar e implementar una infraestructura containerizada (Docker) que permita el despliegue de todos los servicios del sistema.**
- 3. Construir una API REST que exponga los datos procesados y las predicciones del modelo.**
- 4. Desarrollar un portal web accesible para pacientes y personal médico con autenticación segura basada en roles.**
- 5. Definir la estructura de un pipeline de datos (ingesta, procesamiento, calidad y orquestación).**
- 6. Documentar las decisiones técnicas, el proceso de desarrollo con IA y las consideraciones éticas del sistema.**

2. Datos

2.1 Fuentes utilizadas

El dataset utilizado para entrenar el modelo de clasificación de radiografías proviene del conjunto público Chest X-Ray COVID-19 & Pneumonia, disponible en Kaggle.

Este dataset contiene radiografías de tórax etiquetadas en tres categorías: COVID-19, Neumonía y Normal.

Para el entorno de desarrollo y demostración del pipeline, se han generado 300 imágenes sintéticas (100 por clase) almacenadas en `ml/radiology-classifier/data/synthetic/`, distribuidas en las carpetas `COVID-19/`, `NEUMANIA/` y `NORMAL/`. Estas imágenes permiten ejecutar el pipeline completo sin necesidad de descargar el dataset completo.

2.2 Proceso de limpieza y transformación

El pipeline de preprocesamiento, implementado en `ml/radiology-classifier/training/preprocess.py`, aplica las siguientes transformaciones:

Carga y normalización:

- Las imágenes se cargan en escala de grises (canal L), ya que las radiografías son inherentemente monocromáticas.
- Se redimensionan a 224×224 píxeles, tamaño estándar compatible con arquitecturas CNN modernas.
- Los valores de píxel se normalizan del rango 0-255 al rango 0-1 dividiendo entre 255.
- Se convierten a 3 canales (RGB mediante stacking) para compatibilidad con modelos pre-entrenados en ImageNet.

Data Augmentation (solo durante entrenamiento):

- Rotación aleatoria: $\pm 20^\circ$
- Desplazamiento horizontal y vertical: $\pm 20\%$
- Flip horizontal (válido para radiografías de tórax, que son simétricas)

- Variación de brillo: $\pm 20\%$

División del dataset:

- Train: 70% de las muestras (estratificado por clase)
- Validación: 10%
- Test: 20%

La estratificación garantiza que cada partición mantenga la proporción original de clases, evitando sesgos en la evaluación.

3. Arquitectura del sistema

3.1 Diseño del pipeline de datos

El sistema sigue una arquitectura de pipeline por etapas, definida en la carpeta pipelines/:

- **Ingesta:** Ingesta (pipelines/ingestion/): Punto de entrada de radiografías al sistema. En producción implementaría un watcher de carpeta o conector de API hospitalaria (DICOM/HL7).
- **Procesamiento:** Procesamiento (pipelines/processing/): Transformación y normalización de imágenes para su consumo por el modelo.
- **Calidad:** Calidad (pipelines/quality/): Validación de integridad de datos: detección de duplicados, imágenes corruptas o con calidad insuficiente. Los resultados se almacenan en la tabla data_quality_issues de PostgreSQL.
- **Orquestación:** Orquestación (pipelines/orchestration/): Coordinación del flujo completo mediante schedules y triggers.

Los eventos del pipeline se registran en la tabla pipeline_events de la base de datos, lo que permite trazabilidad completa de cada estudio.

3.2 Infraestructura utilizada (Docker, servicios)

Toda la infraestructura está definida en `infra/docker/docker-compose.yml` y se levanta con un único comando. Los servicios desplegados son:

Servicio	Imagen / Tecnología	Puerto	Función
PostgreSQL 16	postgres:16	5432	Base de datos relacional principal
MinIO	minio/minio	9000/9001	Almacenamiento de objetos S3-compatible para imágenes
Mailpit	axllent/mailpit	8025/1025	Servidor SMTP de pruebas (intercepción de emails)
FastAPI (API)	Dockerfile custom	8000	API REST del sistema
pgAdmin 4	dpage/pgadmin4:8	5050	Interfaz de administración de la base de datos
Frontend (nginx)	Dockerfile custom	3000	Portal web estático servido por nginx

Todos los servicios están conectados a la red Docker interna `hospital_net`. Los servicios dependientes esperan a que sus dependencias estén saludables (healthchecks definidos) antes de arrancar.

3.3 Relación entre los distintos componentes

El flujo de datos entre componentes sigue el siguiente orden:

- 1. El usuario accede al frontend (puerto 3000) a través del navegador.**
- 2. El frontend realiza peticiones a la API (puerto 8000) mediante JavaScript.**
- 3. La API autentica al usuario mediante JWT y accede a PostgreSQL para consultar o modificar datos.**
- 4. Las imágenes de radiografías se almacenan en MinIO y la API guarda las referencias (bucket + key) en PostgreSQL.**
- 5. Las predicciones del modelo ML se guardan en la tabla predictions, vinculadas a cada estudio.**

6. Los eventos de SMTP (recuperación de contraseñas) pasan por Mailpit en entorno de desarrollo.

4. Modelos de Inteligencia Artificial

4.1 Justificación del modelo seleccionado

Se han evaluado dos aproximaciones para la clasificación de radiografías:

Opción A: CNN con Transfer Learning (EfficientNetB4) — descrita en la especificación técnica inicial (SPECIFICATIONS.md). EfficientNetB4 es una arquitectura pre-entrenada en ImageNet con ~17M parámetros, que ofrece el mejor balance entre precisión y eficiencia computacional. Fue la arquitectura de referencia durante el diseño.

Opción B: Pipeline clásico sklearn (implementada) — el modelo final implementado en `ml/radiology-classifier/training/model.py` utiliza un pipeline scikit-learn compuesto por tres etapas:

- **StandardScaler:** Normaliza las features para que PCA funcione correctamente (requiere escala homogénea).
- **PCA (100 componentes):** Reduce la dimensionalidad de ~150.000 features (224x224x3) a 100 componentes principales, preservando la máxima varianza posible.
- **MLPClassifier (256->128->64):** Red neuronal densa con activación ReLU, optimizador Adam, regularización L2 ($\alpha=1e-4$) y early stopping (paciencia=15 iteraciones).

Justificación: El pipeline sklearn se seleccionó por su simplicidad de despliegue (sin dependencias de GPU), velocidad de entrenamiento con datasets pequeños y facilidad de serialización con joblib. La arquitectura CNN con transfer learning se mantiene como objetivo de producción documentado.

Se asigna mayor peso de clase a COVID-19 (1.5 frente a 1.0 para las demás clases), dado que un falso negativo en COVID-19 tiene consecuencias clínicas críticas.

4.2 Proceso de entrenamiento

El entrenamiento (`ml/radiology-classifier/training/train.py`) sigue los siguientes pasos:

- 1. Carga de datos:** El `DataPreprocessor` carga todas las imágenes del directorio `data/synthetic/`, aplica normalización y genera la división `train/val/test`.
- 2. Aplanamiento:** Las imágenes (N, 224, 224, 3) se aplanan a vectores (N, 150.528) para su entrada al pipeline `sklearn`.
- 3. Sample weights:** Se calculan pesos por muestra usando `compute_sample_weight` para compensar el desbalance de clases.
- 4. Ajuste del pipeline:** Se ejecuta `pipeline.fit()`, que internamente aplica `Scaler` -> `PCA` -> `MLP` en secuencia.
- 5. Evaluación intermedia:** Se calcula el `accuracy` sobre el conjunto de validación.
- 6. Persistencia:** El modelo final se serializa con `joblib.dump` en `ml/radiology-classifier/models/model_final.pkl` junto a un JSON de metadatos.

4.3 Evaluación mediante métricas

La evaluación (`ml/radiology-classifier/training/evaluate.py`) calcula las siguientes métricas sobre el conjunto de test:

- **Accuracy global:** Proporción de predicciones correctas sobre el total.
- **Precisión por clase:** $TP / (TP + FP)$ — mide cuántas predicciones positivas son correctas.

- Recall (Sensibilidad) por clase: $TP / (TP + FN)$ — mide cuántos positivos reales se detectan.
- F1-Score por clase: Media armónica de precisión y recall.
- Especificidad por clase: $TN / (TN + FP)$.
- Matriz de confusión: Absoluta y normalizada (visualización con seaborn).
- Curvas ROC y AUC: Una por clase, usando binarización one-vs-rest.

El análisis de errores prioriza los falsos negativos en COVID-19 (FN_COVID), que representan el error clínico más grave.

4.4 Resultados obtenidos

Con el dataset sintético de 300 imágenes, el modelo converge en pocas iteraciones gracias al early stopping. Los artefactos generados se almacenan en `ml/radiology-classifier/models/`:

- `model_final.pkl`: Modelo serializado listo para inferencia.
- `training_history.png`: Curvas de loss y accuracy de validación.
- `confusion_matrix.png`: Matrices absoluta y normalizada.
- `roc_curves.png`: Curvas ROC por clase.
- `evaluation_report.json`: Métricas completas en formato JSON.
- `clinical_analysis.json`: Análisis clínico detallado.

Nota importante: Los resultados sobre datos sintéticos tienen validez limitada. El rendimiento real del modelo solo puede evaluarse sobre datos clínicos reales con validación por radiólogos expertos.

5. Automatizaciones

5.1 Automatizaciones implementadas

El sistema contempla las siguientes automatizaciones, cuya estructura está definida en automation/:

- **Alertas:** Alertas (automation/alerts/): Sistema de alertas ante eventos críticos del pipeline (fallos de ingesta, errores de calidad, predicciones con baja confianza). En el estado actual del proyecto, se simulan mediante logs y, en entornos con SMTP configurado, mediante email.
- **Informes:** Informes automáticos (automation/reports/): Generación periódica de informes de rendimiento del modelo y del pipeline (estructura definida, pendiente de implementación completa).
- **File-mover:** Movimiento de ficheros (automation/file-mover/): Organización automática de radiografías entre carpetas de ingesta, procesamiento y archivo.

La automatización más funcional actualmente es el **envío de emails**: el sistema API implementa un flujo completo de recuperación de contraseña con tokens de un solo uso (TTL configurable, por defecto 30 minutos), usando SMTP real o Mailpit en desarrollo.

5.2 Integración dentro del sistema

Las automatizaciones se integran con el sistema principal a través de:

- La tabla pipeline_events de PostgreSQL, donde se registran todos los eventos del pipeline con su estado (ok, error, warning).
- El servicio de email (SMTP/Mailpit) para notificaciones a usuarios.
- El sistema de healthchecks de Docker Compose, que reinicia automáticamente los contenedores que fallen.

6. Integraciones

6.1 Flujo completo de datos

El flujo de datos de extremo a extremo sigue este recorrido:

- 1. Entrada:** Una radiografía se sube al sistema a través de la API REST, que la almacena en MinIO.
- 2. Registro:** La API crea un registro en la tabla `studies` con la referencia al objeto en MinIO (bucket + key), el `patient_id` asociado y la marca de tiempo.
- 3. Preprocesamiento:** El pipeline carga la imagen desde MinIO, aplica las transformaciones (resize, normalización, stack de canales) y genera el vector de features.
- 4. Inferencia:** El modelo (pkl) recibe el vector aplanado y devuelve las probabilidades para cada clase (sana, neumonía, COVID-19).
- 5. Persistencia:** La predicción se guarda en la tabla `predictions` con las tres probabilidades y la etiqueta predicha.
- 6. Consulta:** El médico o administrador consulta el estudio y su predicción a través del portal web o la API.

6.2 Conexión entre los diferentes módulos

Módulo	Se comunica con	Mecanismo
Frontend (nginx)	API FastAPI	HTTP/JSON (REST)
API FastAPI	PostgreSQL	SQLAlchemy + psycopg
API FastAPI	MinIO	SDK minio-py
API FastAPI	Mailpit/SMTP	smtplib (Python stdlib)
Modelo ML	Sistema de archivos	joblib + PIL
pgAdmin	PostgreSQL	Protocolo nativo PostgreSQL

La autenticación entre todos los módulos se gestiona mediante **JWT (JSON Web Tokens)**: el frontend obtiene un token en el login y lo envía en la cabecera **Authorization: Bearer <token>** en cada petición. La API verifica el token y comprueba el rol del usuario antes de permitir el acceso a cada endpoint.

7. Diario de desarrollo con IA

7.1 Herramientas utilizadas

Durante el desarrollo del proyecto se han utilizado las siguientes herramientas de IA:

- **Claude:** Claude (Anthropic / Claude Code): Asistente principal para generación de código, diseño de arquitectura, depuración y documentación. Utilizado a través de la interfaz CLI de Claude Code directamente en el repositorio.
- **Copilot:** GitHub Copilot: Autocompletado de código en el editor durante la escritura de módulos Python y HTML/JavaScript.

7.2 Ejemplos de prompts

A continuación se presentan ejemplos representativos de los prompts utilizados durante el desarrollo:

Prompt 1 — Diseño del modelo:

"Disenha un pipeline sklearn para clasificar radiografias de torax en 3 clases (COVID-19, Neumonia, Normal). Necesito que incluya normalizacion, reduccion de dimensionalidad con PCA y una MLP. Justifica cada decision tecnica y documenta las limitaciones respecto a una CNN con transfer learning."

Resultado: El asistente generó el módulo `model.py` con el pipeline completo, incluyendo los métodos de justificación que aparecen en `_log_architecture_decisions()`.

Prompt 2 — Infraestructura Docker:

"Crea un docker-compose.yml para un sistema hospitalario con PostgreSQL, MinIO para almacenamiento de imagenes, un servidor SMTP de pruebas (Mailpit), una API FastAPI y un frontend nginx. Configura healthchecks y dependencias entre servicios."

Resultado: El archivo infra/docker/docker-compose.yml con todos los servicios, healthchecks, volúmenes y red hospital_net.

Prompt 3 — Autenticación segura:

"Implementa un sistema de autentificacion JWT para FastAPI con roles (admin, medico, paciente). Incluye registro de usuarios, login, recuperacion de contrasena por email con tokens de un solo uso, y validacion de contrasenas seguras."

Resultado: El módulo services/api/app/auth.py con toda la lógica de autenticación, validación de contraseñas y el flujo de reset de contraseña.

7.3 Problemas encontrados y soluciones

Problema 1 — Compatibilidad de puertos SMTP:

Mailpit escucha en el puerto 1025 internamente, pero el archivo .env.example tenía configurado el puerto 587 (Gmail). Cuando el host SMTP era Mailpit y el puerto era 587, los correos no llegaban.

Solución: Se implementó la función `_resolve_smtp_port()` en auth.py que detecta si el host es Mailpit y fuerza el puerto 1025 automáticamente.

Problema 2 — Dimensionalidad de imágenes con sklearn:

El pipeline sklearn no acepta tensores 4D (N, H, W, C). Las imágenes debían aplanarse antes de pasar al StandardScaler.

Solución: Se añade el paso `X_train_flat = X_train.reshape(len(X_train), -1)` antes del entrenamiento en `train.py` y `evaluate.py`.

Problema 3 — Rutas en Docker Compose:

Al ejecutar docker compose desde el directorio `ml/`, las rutas relativas al `docker-compose.yml` fallaban porque el context de build de los servicios apuntaba a `../.. / services/api`.

Solución: Se documentó explícitamente en el README que Docker Compose debe ejecutarse desde `infra/docker/` con la ruta absoluta al proyecto.

8. Justificaciones técnicas

8.1 Decisiones tomadas durante el desarrollo

Decisión	Justificación
PostgreSQL 16	BD relacional madura con soporte JSONB y transacciones ACID, críticas en entornos médicos.
MinIO (S3-compatible)	Almacenamiento de objetos escalable, API compatible con AWS S3 para facilitar migración a nube.
FastAPI	Framework Python moderno con validación Pydantic, documentación OpenAPI automática y alto rendimiento.
JWT para autenticación	Stateless: sin sesiones en memoria. Escalable horizontalmente. TTL configurable.
Roles (admin, médico, paciente)	Separación de responsabilidades: pacientes ven solo sus estudios; médicos acceden a todos.

Decisión	Justificación
Pipeline sklearn vs CNN	sklearn es más sencillo para demostrar el flujo completo sin GPU; CNN es el objetivo de producción.
Early stopping (paciencia=15)	Previene overfitting automáticamente sin tuning manual del número de epochs.
PCA(100 componentes)	Reduce 150.528 features a 100 componentes, haciendo el entrenamiento tractable en CPU.
class_weight COVID=1.5	Los falsos negativos en COVID-19 tienen mayor coste clínico que los de otras clases.

8.2 Alternativas consideradas

- **MongoDB vs PostgreSQL:** MongoDB vs PostgreSQL: Se descartó MongoDB porque el modelo de datos es fundamentalmente relacional (pacientes -> estudios -> predicciones) y las transacciones ACID son críticas en entornos médicos.
- **Redis vs JWT:** Redis para sesiones vs JWT: Se descartó Redis porque JWT stateless es suficiente para el alcance del proyecto y evita añadir otro servicio.
- **Streamlit vs HTML:** Streamlit como dashboard vs HTML estático: Se optó por HTML/JS puro con nginx para mayor control sobre la interfaz y sin dependencia de un proceso Python en ejecución.
- **ResNet50 vs EfficientNetB4:** ResNet50 vs EfficientNetB4: EfficientNetB4 tiene mejor ratio precisión/parámetros que ResNet50 para el tamaño de entrada 224×224.

9. Reflexión crítica

9.1 Limitaciones del sistema

El sistema presenta las siguientes limitaciones:

- **Dataset sintético:** Dataset sintético: Las 300 imágenes generadas para la demo no representan la distribución real de casos clínicos. El modelo entrenado sobre ellas no puede aplicarse en un entorno hospitalario real.

- **Caja negra:** Modelo caja negra: Ni el pipeline sklearn (con PCA) ni una CNN explican su razonamiento. No es posible saber qué regiones de la radiografía han determinado la predicción.
- **Sin validación:** Sin validación clínica: El modelo no ha sido revisado por ningún radiólogo. La evaluación sobre datos sintéticos no garantiza generalización.
- **Pipeline incompleto:** Pipeline parcialmente implementado: Las carpetas de pipelines/, automation/ y docs/ están estructuradas pero la mayoría de su contenido son placeholders.
- **Sin monitorización:** Sin monitorización en producción: No hay sistema de alertas ante degradación del modelo (model drift), esencial en entornos donde las condiciones epidemiológicas cambian.

9.2 Posibles mejoras

1. **Transfer Learning real:** Sustituir el pipeline sklearn por EfficientNetB4 fine-tuneado con datos reales (COVIDx, CheXpert).
2. **Explicabilidad:** Implementar GradCAM (Gradient-weighted Class Activation Mapping) para visualizar qué zonas de la radiografía activan cada clase.
3. **Dataset real:** Usar el dataset completo de Kaggle u otros repositorios clínicos validados.
4. **Monitorización del modelo:** Implementar alertas de degradación de rendimiento (data drift, concept drift).
5. **Orquestación real:** Implementar los DAGs de Airflow o similar en pipelines/ orchestration/.
6. **Tests de integración:** Ampliar la cobertura de tests en tests/ para cubrir el pipeline completo.
7. **Dashboard de métricas:** Integrar Grafana o Streamlit para visualizar el rendimiento del sistema en tiempo real.

9.3 Aplicación en un entorno real

Para desplegar este sistema en un hospital real se requerirían los siguientes pasos adicionales:

1. **Certificación regulatoria:** Obtener el marcado CE como producto sanitario de clase IIa (Reglamento EU 2017/745) y, si aplica, aprobación FDA 510(k).
2. **Dataset real validado:** Entrenar con datos de pacientes reales con consentimiento informado, con al menos 10.000 imágenes por clase.
3. **Integración DICOM/HL7:** Conectar el sistema con los PACS hospitalarios y los sistemas HIS/RIS existentes.
4. **Validación clínica prospectiva:** Ejecutar un ensayo clínico controlado comparando el rendimiento del modelo con el de radiólogos expertos.
5. **Formación del personal:** Protocolo de uso responsable para evitar la sobredependencia del sistema.

10. Consideraciones éticas y legales

10.1 Sesgos en los datos o modelos

El modelo podría presentar los siguientes sesgos:

- **Género:** Sesgo de género: Si el dataset de entrenamiento no está equilibrado por género, el modelo puede tener un rendimiento inferior para el grupo subrepresentado.
- **Edad:** Sesgo de edad: La presentación radiológica de la neumonía difiere entre niños, adultos y mayores. Un dataset no estratificado puede generar predicciones incorrectas en grupos etarios extremos.
- **Equipamiento:** Sesgo de equipamiento: Diferentes marcas y modelos de equipos de rayos X producen imágenes con características técnicas distintas. Un modelo entrenado en una sola marca puede fallar con otra.

- **Geografía:** Sesgo geográfico: El COVID-19 presenta diferentes variantes según la región. Un modelo entrenado en datos de un solo país puede no generalizar.
- **Prevalencia:** Sesgo de prevalencia: Si la proporción de clases en el dataset de entrenamiento no refleja la prevalencia real en la población objetivo, las probabilidades predichas estarán sesgadas.

Mitigación: Auditoría de sesgo pre-despliegue, validación en múltiples hospitales y grupos demográficos, y monitorización continua post-despliegue.

10.2 Privacidad

Las radiografías de tórax son datos de salud especialmente protegidos, clasificados como categoría especial según el RGPD (Reglamento General de Protección de Datos, art. 9). Su tratamiento exige:

- **Base legal:** Base legal explícita: Consentimiento informado del paciente o habilitación legal específica.
- **Minimización:** Minimización de datos: Solo recopilar los datos estrictamente necesarios para el diagnóstico.
- **Seudonimización:** Seudonimización: Los identificadores de paciente deben separarse de las imágenes médicas en capas de almacenamiento distintas.
- **Derechos ARCO:** Derecho de acceso y supresión: Los pacientes deben poder solicitar el borrado de sus datos.
- **DPIA:** Evaluación de impacto (DPIA): Obligatoria antes del despliegue, dado que el tratamiento masivo de datos biométricos constituye un riesgo alto.

En el sistema actual, la tabla patients almacena datos personales (nombre, teléfono, fecha de nacimiento) mientras que studies almacena las referencias a las imágenes. Esta separación es un primer paso hacia laseudonimización.

10.3 Riesgos del sistema

Riesgo	Probabilidad	Impacto	Mitigación
Falso negativo COVID-19	Media	Crítico	Umbral ajustado, revisión humana obligatoria
Sobredependencia del personal	Alta	Alto	Formación, disclaimer en la interfaz
Filtración de datos de pacientes	Baja	Crítico	Cifrado TLS/HTTPS, control de acceso por roles
Degradación del modelo (drift)	Alta a largo plazo	Alto	Monitorización continua, reentrenamiento periódico
Uso fuera del contexto validado	Media	Alto	Documentar limitaciones en la interfaz

Conclusión ética: Este sistema, en su estado actual, es un prototipo académico y no debe utilizarse para tomar decisiones clínicas reales. Cualquier uso en entorno hospitalario requiere validación clínica rigurosa, certificación regulatoria y un marco de gobernanza que garantice la supervisión humana de todas las predicciones del modelo.